

Resolução dos exercícios — Aula 008

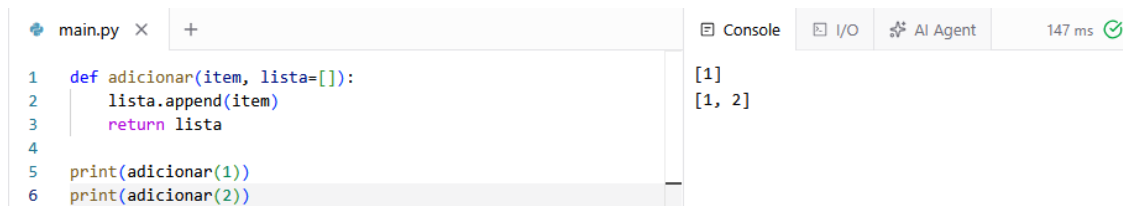
1. Python

```
def adicionar(item, lista=[]):  
    lista.append(item)  
    return lista
```

```
print(adicionar(1))  
print(adicionar(2))
```

a) Saída:

```
[1]  
[1, 2]
```



The screenshot shows a code editor with a file named 'main.py'. The code defines a function 'adicionar' that takes an 'item' and a default empty list 'lista'. It appends the item to the list and returns it. Below the function, there are two print statements: 'print(adicionar(1))' and 'print(adicionar(2))'. To the right of the code editor, there is a 'Console' panel showing the output: '[1]' and '[1, 2]'. Above the console, there are tabs for 'I/O', 'AI Agent', and a timer showing '147 ms' with a green checkmark.

Saída py

b) Conceito: o valor padrão mutável (`lista=[]`) é criado uma única vez e reutilizado nas chamadas seguintes.

2. Java

```
static void zera(int[] v, int n) {  
    v[0] = 0;  
    n = 0;  
}
```

```
int[] v = {5, 5};  
int n = 5;  
zera(v, n);  
System.out.println(v[0] + " " + n);
```

a) Saída:

```
0 5
```

```
J Main.java x + Console I/O AI Agent 975 ms ✓
1 import java.util.*;
2
3 public class Main {
4     static void zera(int[] v, int n) {
5         v[0] = 0;
6         n = 0;
7     }
8
9     public static void main(String[] args) {
10
11
12
13     int[] v = {5, 5}; int n = 5;
14     zera(v, n);
15     System.out.println(v[0] + " " + n);
16
17 }
18 }
```

Saída java

b) Conceito: o array é um objeto compartilhado, então sua posição é alterada; o inteiro é passado por valor e a atribuição em n não altera a variável original.

3. Python

```
fs = [lambda: i for i in range(3)]
print([f() for f in fs])
```

a) Saída:

[2, 2, 2]

```
main.py x + Console I/O AI Agent 144 ms ✓
1 fs = [lambda: i for i in range(3)]
2 print([f() for f in fs])
```

Saída py

b) Conceito: as funções capturam a variável i, e não uma cópia do valor em cada iteração. Quando são executadas, i já vale 2.

4. C

```
int contador(void) {
    static int n = 0;
    return ++n;
}
```

```
// em main:
contador();
contador();
printf("%d\n", contador());
```

a) Saída:

3

```
C Main.c x + Console I/O AI Agent 193 ms
1  #include <stdio.h>
2
3  int contador(void) {
4      static int n = 0;
5      return ++n;
6  }
7
8  int main()
9  {
10     // em main:
11     contador(); contador();
12     printf("%d\n", contador());
13 }
14
```

Saída c

b) Conceito: uma variável local static mantém seu valor entre as chamadas da função.

5. Rust

```
fn dobra(v: Vec<i32>) -> Vec<i32> {
    v.iter().map(|x| x * 2).collect()
}
```

```
let v = vec![1, 2, 3];
let d = dobra(v);
println!("{:?} {:?}", v, d);
```

a) Saída:

```
error[E0382]: borrow of moved value: `v`
--> HelloWorld.rs:8:27
```

```
6 |     let v = vec![1, 2, 3];
   |         - move occurs because `v` has type `Vec<i32>`, which does not
   |         implement the `Copy` trait
7 |     let d = dobra(v);
   |               - value moved here
8 |     println!("{:?} {:?}", v, d);
   |                          ^ value borrowed here after move
```

note: consider changing this parameter type in function `dobra` to borrow instead if owning the value isn't necessary

```
--> HelloWorld.rs:1:13
```

```
1 | fn dobra(v: Vec<i32>) -> Vec<i32> {
   |     ----- ^^^^^^^^^ this parameter takes ownership of the value
   |     |
   |     in this function
```

help: consider cloning the value if the performance cost is acceptable

```

7 |         let d = dobra(v.clone());
          ++++++++

```

error: aborting due to 1 previous error

For more information about this error, try `rustc --explain E0382`.

```

HelloWorld.rs X
1 fn dobra(v: Vec<i32>) -> Vec<i32> {
2     v.iter().map(|x| x * 2).collect()
3 }
4
5 fn main() {
6     let v = vec![1, 2, 3];
7     let d = dobra(v);
8     println!("{:?} {:?}", v, d);
9 }
10

error[E0382]: borrow of moved value: `v`
--> HelloWorld.rs:8:27
|
6 |     let v = vec![1, 2, 3];
|         - move occurs because `v` has type `Vec<i32>`, which does not implement the `Copy` trait
7 |     let d = dobra(v);
|         - value moved here
8 |     println!("{:?} {:?}", v, d);
|                           ^ value borrowed here after move
|
note: consider changing this parameter type in function `dobra` to borrow instead if owning the value isn't necessary
--> HelloWorld.rs:1:13
1 | fn dobra(v: Vec<i32>) -> Vec<i32> {
|     ----- ^^^^^^^ this parameter takes ownership of the value
|     |
|     in this function
help: consider cloning the value if the performance cost is acceptable
7 |     let d = dobra(v.clone());
|                   ++++++++

error: aborting due to 1 previous error

For more information about this error, try `rustc --explain E0382`.

```

Saída rs

O compilador acusa o uso de v depois de ele ter sido movido para dobra(v).

b) Conceito: ownership e movimento (*move*). Ao passar v por valor para dobra, a função assume a posse do vetor e v deixa de poder ser usado no escopo original.

6. Python

```
total = 0
```

```

def adiciona(x):
    total = total + x
    return total

```

```
print(adiciona(5))
```

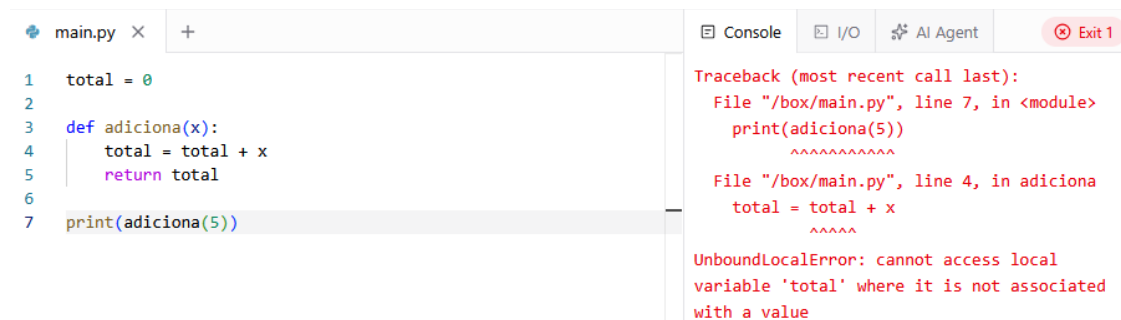
a) Saída:

Traceback (most recent call last):

```
File "/box/main.py", line 7, in <module>
    print(adiciona(5))
    ^^^^^^^^^^^^^
```

```
File "/box/main.py", line 4, in adiciona
    total = total + x
    ^^^^^
```

UnboundLocalError: cannot access local variable 'total' where it is not associated with a value



The screenshot shows a code editor with a file named 'main.py'. The code in the editor is as follows:

```
1 total = 0
2
3 def adiciona(x):
4     total = total + x
5     return total
6
7 print(adiciona(5))
```

On the right side of the editor, there is a 'Console' tab showing the following error message:

```
Traceback (most recent call last):
  File "/box/main.py", line 7, in <module>
    print(adiciona(5))
    ^^^^^^^^^^^^^
  File "/box/main.py", line 4, in adiciona
    total = total + x
    ^^^^^
UnboundLocalError: cannot access local
variable 'total' where it is not associated
with a value
```

Saída py

O Python considera `total` local à função por causa da atribuição, mas tenta ler essa variável antes de ela receber um valor local.

b) Conceito: escopo léxico e sombreamento de variáveis. Para alterar o `total` global, seria necessário declarar `global total` dentro da função.